Basic (your first real API call) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The three pieces, and what each returns
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The three pieces, and what each returns

Basic Version

You need three things: a client (your connection to OpenAI), a list of messages (what you're saying to it), and one line to print the reply.

The client reads your API key automatically from the `OPENAI_API_KEY` environment variable, if `.env` is already loaded — you don't have to type the key into your code anywhere.

The reply text is buried a few levels deep in the response object: `response.choices[0].message.content`. There's always at least one item in `choices`, so `[0]` is safe here.

Pick two very different system prompts to compare — like "You are a formal, professional assistant" versus "You are a sarcastic pirate" — so the change in the answer is obvious.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Three pieces, matching the Core Concepts above: `client = OpenAI()` creates the client (it reads `OPENAI_API_KEY` from the environment on its own, so call `load_dotenv()` from Doc01's config pattern first if you haven't already). `client.chat.completions.create(model=..., messages=[...])` sends the request. The `messages` list needs at least one `{"role": "system", ...}` entry and one `{"role": "user", ...}` entry — the system entry sets behavior, the user entry is the actual question.

`response.choices` is a list because the API can technically return more than one candidate answer (controlled by an `n` parameter you're not using here) — with the default settings, there's exactly one, at index `0`. `response.choices[0].message.content` is a plain string: the model's reply text.

For the system-prompt comparison, keep the user message identical across both runs and change only the system message — that isolates the variable you're actually testing.

Difference between Basic and Intermediate: Basic names the three pieces and where the reply text lives, for the tidy one-shot case. Intermediate explains exactly what each piece is doing and why `choices` is a list at all.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
import the OpenAI client
load the .env file

make the client

messages = [
    system message: "You are a formal, professional assistant."
    user message: "Tell me about your day."
]

send the messages to the model
print the reply text

change the system message to something very different
send again with the same user message
print the new reply text
```

Here's almost the whole thing for the first call — just try running it and reading it line by line:

```
# chat_api_basics_practice.py
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are formal and professional."},
        {"role": "user", "content": "Tell me about your day."},
    ],
)
print(response.choices[0].message.content)
```

Expected output if you run just this: a formal, professional-sounding reply about "having a day," since language models play along with the premise. What's missing: loading `.env` first, and the

second call with a different system prompt.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

function ask(system_prompt, user_prompt) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    return response.choices[0].message.content

print(ask("You are formal and professional.", "Tell me about your day."))
print(ask("You are a sarcastic pirate.", "Tell me about your day."))
```

Wrapping the call in a small `ask()` function is optional for this exercise but makes comparing two system prompts much less repetitive. Write it this way, run it, confirm the two replies clearly differ in tone, then compare against the Solution.

Then move each system prompt into a file (`prompts/formal.txt`) and write `load_prompt(name)` : `with open(path) as prompt_file:` then `return prompt_file.read().strip()` . Use `name.lower()` in the path, so `"Formal"` still finds `formal.txt` .

Difference between Basic and Intermediate: Basic's near-complete code makes one call and prints the reply, with nothing shared between calls. Intermediate wraps the same call in a reusable `ask()` function, so comparing prompts is a one-liner instead of a copy-pasted block.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution